

Documentation développeur

Fonctionnement interne de la version 2.0.4

Version 2.0.4 - exemples anonymisés

1. Objectif

Ce document explique le fonctionnement interne de l'adaptateur Trax ILIAS Bridge 2.0.4, en particulier `aggregate.php`, la configuration Apache et `config.php`. Il est destiné aux développeurs et mainteneurs qui doivent auditer, corriger ou faire évoluer l'adaptateur.

2. Fichiers principaux

Fichier	Responsabilité
<code>aggregate.php</code>	Front controller PHP ; routage, nettoyage de requêtes, proxy HTTP et agrégation compatible ILIAS.
<code>config.sample.php</code>	Modèle de configuration applicative.
<code>apache-trax-iliad-adapter.conf</code>	Exemple Apache simple avec <code>AliasMatch</code> globaux.
<code>apache-trax-iliad-adapter-https-internal.example.conf</code>	Exemple Apache avec vhost public et backend interne.
<code>README.md</code>	Documentation d'installation et d'exploitation.
<code>CHANGELOG.md</code>	Historique des versions.

3. Routage dans `aggregate.php`

Le script lit `REQUEST_URI`, extrait le chemin et la méthode HTTP, puis applique quatre motifs de routage.

```
/cmi5/tokens          -> handleSanitizedProxyRequest()
/xapi/activities/state -> handleSanitizedProxyRequest()
/api/statements/aggregate -> handleStatementsAggregate()
/stores/api/statements/aggregate sans store -> handleStatementsAggregate() avec résolution du store
```

Si aucune route ne correspond, l'adaptateur renvoie un JSON 404 `route_not_handled`.

4. `handleSanitizedProxyRequest()`

Cette fonction est utilisée pour les flux `cmi5` où ILIAS peut envoyer le paramètre non standard `activity_id`. Elle supprime `activity_id` de la query string et du corps de requête, mais ne supprime pas `activityId`.

Étape	Détail
Lecture des paramètres	Copie de <code>\$_GET</code> puis <code>unset(activity_id)</code> .
Lecture du corps	<code>php://input</code> est analysé selon le <code>Content-Type</code> .
Nettoyage JSON	Suppression de la clé <code>activity_id</code> si le corps est JSON.
Nettoyage formulaire	Suppression de la clé <code>activity_id</code> si le corps est <code>x-www-form-urlencoded</code> .
Construction URL	<code>buildBaseUrl(config) + path + query</code> corrigée.
En-têtes	Authorization, Accept, Content-Type et X-Experience-API-Version sont transmis.
Proxy	<code>httpRequest()</code> appelle Trax puis <code>relayResponse()</code> renvoie la réponse.

```
$queryParams = $_GET;
unset($queryParams['activity_id']);

$rawBody = file_get_contents('php://input') ? : '';
```

```
$body = cleanRequestBody($rawBody, $contentType);

$targetUrl = buildBaseUrl($config) . $path;
```

5. cleanRequestBody()

La fonction nettoie activity_id dans trois cas : JSON, formulaire URL-encodé et fallback conservateur.

Content-Type	Comportement
application/json	json_decode, unset(activity_id), json_encode.
application/x-www-form-urlencoded	parse_str, unset(activity_id), http_build_query.
Autre texte assimilable à formulaire	parse_str et suppression si activity_id est détecté.
Corps non reconnu	Transmission du corps original.

6. handleStatementsAggregate()

ILIAS appelle une route d'agrégation, mais Trax expose les statements via l'endpoint xAPI standard. La fonction convertit donc l'appel aggregate en lecture /xapi/statements, puis applique localement les étapes de pipeline utiles à ILIAS.

```
GET /trax/api/gateway/clients/{client}/stores/{store}/api/statements/aggregate?pipeline=...
-> GET {trax_base_url}/trax/api/gateway/clients/{client}/stores/{store}/xapi/statements?limit=5000
-> aggregateForIlias(statements, pipeline)
-> JSON compatible ILIAS
```

L'en-tête X-Experience-API-Version est fixé à 1.0.3 pour éviter les doublons envoyés à Trax.

7. aggregateForIlias()

La fonction simule une partie de la logique d'agrégation de type MongoDB attendue par ILIAS.

Stage supporté	Usage
\$match	Filtrer les statements par champs, verbes, activité, score, etc.
\$sort	Trier les statements avant affichage ou regroupement.
\$skip	Pagination.
\$limit	Limite de résultats.
\$group	Regrouper par acteur pour Ranking ou autres rapports.
\$facet	Réponse spéciale pour les tables Learning Experiences avec maxcount et statements.

Chaque statement est encapsulé dans deux formes pour maximiser la compatibilité :

```
$docs[] = ['statement' => $s] + $s;
```

Ainsi, les chemins statement.verb.id et verb.id peuvent tous deux être résolus selon le pipeline reçu.

8. matchesCriteria() et pathValues()

matchesCriteria() évalue les filtres de type \$match. pathValues() permet d'accéder à des chemins pointés comme statement.result.score.scaled, même dans des tableaux imbriqués.

Opérateur	Comportement
\$exists	Vérifie la présence ou absence d'un champ.
\$in / \$nin	Teste l'appartenance à une liste.
\$eq / \$ne	Égalité ou différence.
\$gte / \$gt / \$lte / \$lt	Comparaisons numériques ou lexicales simples.
\$regex	Recherche par expression régulière insensible à la casse.
\$and / \$or	Combinaison de sous-conditions.

9. groupDocs() et Ranking

Ranking ILIAS utilise généralement un pipeline qui filtre sur `statement.result.score.scaled`, trie par timestamp, puis groupe par acteur. `groupDocs()` calcule les accumulateurs demandés et sécurise les lignes retournées.

- Les groupes sans identifiant utilisateur exploitable sont ignorés.
- `account` peut être dérivé de `_id` si le pipeline groupe par `actor.account.name`.
- `timestamp` est normalisé si une ligne contient un score ou un champ timestamp.
- Les lignes avec score absent ou `account` vide sont ignorées pour éviter les erreurs côté ILIAS.

10. Normalisation du timestamp

ILIAS peut appeler `getTimestamp()` sur la date retournée par la ligne Ranking. Si le timestamp est absent ou non parsable, l'interface peut planter. La version 2.0.4 normalise donc les dates en UTC avec millisecondes.

```
normalizeXapiTimestamp($value)
  -> DateTimeImmutable($value)
  -> UTC
  -> format Y-m-dTH:i:s.vZ
```

Ordre de fallback :

1. `row.timestamp` issu du pipeline.
2. `statement.timestamp` du dernier statement du groupe.
3. timestamp direct si disponible.
4. `statement.stored` ou `stored`.
5. date UTC courante en dernier recours.

11. buildBaseUrl()

`buildBaseUrl()` détermine l'URL cible Trax. Si `trax_base_url` est renseigné, il est utilisé tel quel après suppression du slash final. Sinon, le script reconstruit l'URL à partir de HTTPS, X-Forwarded-Proto et HTTP_HOST.

Recommandation : en HTTPS avec nom public, renseigner explicitement `trax_base_url` vers un backend interne, par exemple `http://127.0.0.1:8080`, afin d'éviter toute boucle.

12. forwardHeaders()

La fonction ne transmet qu'une liste blanche d'en-têtes afin d'éviter des effets de bord.

Route	En-têtes transmis
Proxy cmi5/state	Authorization, Accept, Content-Type, X-Experience-API-Version.
Aggregate	Authorization, Accept, puis X-Experience-API-Version: 1.0.3 ajouté explicitement.

La récupération de `Authorization` couvre `HTTP_AUTHORIZATION`, `REDIRECT_HTTP_AUTHORIZATION` et `PHP_AUTH_USER/PHP_AUTH_PW`.

13. httpRequest() et relayResponse()

`httpRequest()` utilise cURL avec `CURLOPT_FOLLOWLOCATION` à false. Ce choix évite de suivre automatiquement une redirection interne, ce qui rend les erreurs visibles et facilite le diagnostic. `relayResponse()` renvoie le code HTTP, les en-têtes utiles et le corps au client ILIAS.

- Timeout de connexion : 10 secondes.
- Timeout global : 60 secondes.
- Suppression des en-têtes hop-by-hop : transfer-encoding, content-length, connection, server, date.
- Ajout d'en-têtes de diagnostic : X-Trax-Ilias-Bridge-Bypass et X-Trax-Ilias-Bridge-Version.

14. Configuration Apache développeur

Règles 2.0.4 à maintenir :

```
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/api/statements/aggregate$" "/var/www/trax-ilias-
aggregate-adapter/aggregate.php"
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/api/statements/aggregate$" "/var/www/trax-ilias-
aggregate-adapter/aggregate.php"
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/cmi5/tokens$" "/var/www/trax-ilias-aggregate-
adapter/aggregate.php"
AliasMatch "^/trax/api/gateway/clients/[^/]+/stores/[^/]+/xapi/activities/state$" "/var/www/trax-ilias-
aggregate-adapter/aggregate.php"
```

Important : ne pas intercepter /xapi/statements ni /xapi/about, afin de ne pas perturber les flux xAPI standards non-cmi5.

15. Configuration config.sample.php

Paramètre	Type	Valeur typique	Impact
trax_base_url	string null	http://127.0.0.1:8080	URL backend Trax utilisée par le proxy.
aggregate_store_strategy	string	client_name	Résolution du store quand l'URL ILIAS l'omet.
client_store_map	array	['eform'=>'default']	Exceptions client -> store.
default_store	string	default	Fallback si stratégie non client_name.
max_statements_to_fetch	int	5000	Limite de lecture xAPI pour aggregate.
debug	bool	false	Expose ou masque les détails d'erreur.

16. Évolutions 2.0.4 par rapport à 2.0.3

- Ajout du routage /xapi/activities/state.
- Utilisation de handleSanitizedProxyRequest() pour /cmi5/tokens et /xapi/activities/state.
- Nettoyage de activity_id dans la query string et dans les corps JSON/formulaire.
- Exemple Apache avec vhost interne sur 127.0.0.1:8080.
- Normalisation des timestamps dans les lignes de Ranking.
- Filtrage des lignes Ranking incomplètes pour éviter des erreurs dans ILIAS.

17. Tests développeur recommandés

```
php -l aggregate.php
php -l config.sample.php
```

```
curl -i http://127.0.0.1:8080/trax/api/gateway/clients/eform/stores/default/xapi/about
```

```
curl -k -i
https://trax.example.org/trax/api/gateway/clients/eform/stores/default/cmi5/tokens?activity_id=test
```

```
curl -k -s -u "eform:password" -H "X-Experience-API-Version: 1.0.3"  
"https://trax.example.org/trax/api/gateway/clients/eform/stores/default/xapi/statements?limit=20"
```

18. Règles de non-régression

- Une ressource cmi5 doit produire au moins launched, initialized et terminated lorsque le contenu se termine correctement.
- LMS.LaunchData doit passer en PUT 204 puis GET 200.
- Les routes /xapi/statements et /xapi/about ne doivent pas passer dans l'adaptateur.
- Learning Experiences doit retourner des données en 200 via aggregate.
- Ranking doit rester vide sans score global, mais ne doit pas provoquer d'erreur PHP ILIAS.
- Un statement avec result.score.scaled et timestamp valide doit permettre l'affichage du Ranking.